

LLM Efficiency Gateway

AI Cost and Token Optimizer Project Brief

One-line description

An AI middleware system that reduces LLM token usage, estimated API cost, and latency using semantic caching, prompt compression, RAG context pruning, dynamic model routing, and quantized local models while preserving answer quality.

Main Problem

Modern AI applications are becoming expensive because they send too many unnecessary tokens to large models. Many applications use long prompts, repeated conversation history, oversized RAG chunks, and expensive models even for simple tasks. This increases company bills, latency, and infrastructure load.

The project targets a real 2026 AI engineering problem: making LLM systems cheaper, faster, and more reliable without simply reducing answer quality.

What The Project Does

Token counting	Measures input tokens, output tokens, expected token budget, and estimated cost before and after optimization.
Semantic caching	Reuses prior answers when a new query is semantically similar enough, avoiding unnecessary LLM calls.
Prompt compression	Removes duplicate instructions, irrelevant chat history, redundant examples, and low-value context.
RAG optimization	Retrieves candidate chunks, reranks them, and sends only the most useful context instead of flooding the model.
Model routing	Sends simple requests to small or local models and reserves stronger models for complex reasoning tasks.
Quantized local inference	Runs small tasks such as classification, routing, query rewriting, and summarization on 4-bit or 8-bit local models.
Quality tracking	Compares optimized answers with baseline answers using semantic similarity, benchmark labels, or human review.
Dashboard	Shows token savings, estimated cost savings, latency, cache hit rate, model usage, and quality retention.

System Workflow

User Query

- > Token Counter
- > Semantic Cache Check
- > Query Complexity Classifier
- > RAG Retrieval + Reranking
- > Prompt Compression
- > Model Router
- > LLM / Local Quantized Model
- > Quality Evaluation
- > Cost + Token Dashboard

Core Tech Stack

Layer	Recommended Stack	Purpose
Backend	Python, FastAPI	Expose optimization gateway APIs and benchmark endpoints.
Workflow	LangGraph, LangChain	Build controllable agent and routing workflows.
RAG	FAISS / Chroma / Qdrant, BM25, rerankers	Retrieve and prune context efficiently.
Models	Llama, Mistral, Qwen	Use open-source models for local and free testing.
Fine-tuning	LoRA / QLoRA	Fine-tune router, compressor, or quality classifier.
Quantization	4-bit / 8-bit quantization	Reduce memory and run lightweight local inference.
Measurement	tiktoken, HF tokenizers, pricing config	Estimate API cost without spending money.
Evaluation	RAGAS, DeepEval, semantic similarity	Measure quality, hallucination risk, and retained performance.
Frontend	Streamlit or Next.js	Visualize savings, latency, cache usage, and quality.
Storage	SQLite / PostgreSQL	Store traces, cache entries, benchmark runs, and metrics.
Deployment	Docker, GitHub, optional Hugging Face Spaces	Make the project reproducible and demo-ready.

Two Extra Stacks Worth Learning

vLLM	Use it for high-throughput LLM serving, batching, streaming, and realistic inference benchmarking.
Triton / TensorRT-LLM	Use one of these if you want a deeper systems angle around inference optimization and deployment performance.

Practical alternative: learn vLLM plus LiteLLM if you want faster implementation. LiteLLM is useful for multi-model routing, provider abstraction, and cost tracking.

How To Measure Cost Without Paying

You can estimate cost by counting tokens locally and applying a pricing configuration. This is honest and recruiter-friendly as long as the project clearly says estimated API cost.

```
estimated_cost =
(input_tokens / 1_000_000) * model_input_price
+ (output_tokens / 1_000_000) * model_output_price
```

Input tokens saved	Count tokens before and after prompt compression and RAG pruning.
Output tokens saved	Count final response tokens from baseline and optimized flows.
Estimated cost saved	Apply the same pricing config to both baseline and optimized requests.
Cache savings	Track how many LLM calls were skipped because semantic cache answered the request.
Latency	Time each pipeline stage and compare baseline versus optimized workflow.
Quality retained	Use exact match, semantic similarity, evaluator model scores, or human scoring.

Example Benchmark Result

Metric	Baseline RAG Agent	Optimized Gateway
Average input tokens	12,000	3,800
Average output tokens	1,200	900
Estimated cost/request	\$0.16	\$0.05
Latency	9.5 sec	4.2 sec
Quality score	86%	84%
Final result	No optimization	68% token reduction, 69% estimated cost reduction, 2% quality drop

Best Resume Description

Resume bullet

Built an LLM Efficiency Gateway that reduced estimated LLM API cost by 60-70% using semantic caching, prompt compression, dynamic model routing, RAG context pruning, quantized local inference, and quality-aware evaluation.

Why This Project Is Impressive

This does not look like a normal chatbot. It shows that you understand LLMs, RAG, agents, fine-tuning, quantization, and the production problems companies care about: cost, latency, scalability, reliability, and evaluation.

The strongest version of this project should include a clean GitHub README, reproducible benchmarks, screenshots of the dashboard, and a short demo video showing baseline versus optimized requests.